AI-POWERED AUTOMOBILE KNOWLEDGE ASSISTANT

Retrieval-Augmented Generation (RAG) Based Mini Project

PROJECT DOCUMENTATION

Project Type: Mini Project

Domain: Artificial Intelligence / Generative AI / RAG

Primary Purpose: Automobile Knowledge Retrieval and Question Answering

Proposed Frontend: React + Vite

Proposed Backend: FastAPI

Database: PostgreSQL + pgvector

LLM: Google Gemini

Prepared for Academic Demonstration

TABLE OF CONTENTS

1. 1. Abstract
2. 2. Introduction
3. 3. Problem Statement
4. 4. Existing System
5. 5. Proposed System
6. 6. Objectives
7. 7. Scope of the Project
8. 8. Key Features
9. 9. System Architecture
10. 10. Technology Stack
11. 11. Project Modules
12. 12. Database Design
13. 13. RAG Workflow
14. 14. Document Management Workflow
15. 15. Search and Question Answering Workflow
16. 16. Frontend Design and Current Interface
17. 17. API Design
18. 18. Security and Data Protection
19. 19. Error Handling
20. 20. Testing Strategy
21. 21. Expected Results
22. 22. Advantages
23. 23. Limitations
24. 24. Future Enhancements
25. 25. Deployment Plan
26. 26. Project Structure
27. 27. Conclusion
28. 28. Viva Questions and Answers

1. ABSTRACT

The AI-Powered Automobile Knowledge Assistant is a Retrieval-Augmented Generation (RAG) based application designed to answer automobile-related questions using information stored in uploaded documents. The system allows users to upload automobile PDFs, process their content, create searchable knowledge chunks, generate embeddings, retrieve relevant information, and obtain grounded answers from a Large Language Model (LLM). Instead of relying only on the general knowledge of the language model, the application first searches the project knowledge base and supplies relevant document content to the LLM. The generated response is accompanied by source references such as document name and page number, making the answer easier to verify.

The proposed architecture uses React and Vite for the frontend, FastAPI for backend services, PostgreSQL with pgvector as the single persistent database, Hugging Face sentence-transformer embeddings for semantic search, a CrossEncoder for reranking, and Google Gemini for answer generation. Firebase Authentication can be used for secure user login. The system is designed as a modular mini project that can later be extended to support more automotive documents, multilingual queries, voice interaction, advanced analytics, and cloud deployment.

2. INTRODUCTION

Automobile information is distributed across service manuals, maintenance guides, technical documents, user manuals, specification sheets, and other reference materials. Finding a particular piece of information manually can be slow, especially when several documents are involved. A conventional chatbot may also produce answers that are not directly supported by the user's documents.

Retrieval-Augmented Generation addresses this problem by combining information retrieval with generative AI. The retrieval component searches the knowledge repository for relevant passages, while the generative component uses those passages as context to produce a natural-language response. This project applies the approach to an automobile knowledge domain.

3. PROBLEM STATEMENT

To design and develop an AI-powered automobile knowledge assistant that can ingest automobile documents, build a searchable knowledge repository, retrieve relevant information for user queries, generate context-grounded answers using a Large Language Model, and provide source references for verification.

4. EXISTING SYSTEM

- Manual searching through automobile manuals and PDF documents.
- Keyword-only search may miss semantically related information.
- General-purpose chatbots may provide answers without using the user's specific documents.
- Users may have difficulty identifying the exact page or document supporting an answer.

- Separate tools are often required for document storage, search, and question answering.

These limitations motivate an integrated document-to-answer RAG system.

5. PROPOSED SYSTEM

The proposed system provides a unified interface for document management, knowledge retrieval, and AI question answering. Automobile documents are uploaded through the web interface and processed by the backend. Text is extracted from each page, OCR is used when required, and the content is divided into searchable chunks. Embeddings are generated for the chunks and stored with the document information in PostgreSQL using pgvector.

When the user asks a question, the system performs semantic and keyword retrieval, combines the candidate results, reranks them, and passes the most relevant context to Gemini. The final answer is displayed together with real database sources. Conversations are stored so users can review previous questions and answers.

6. OBJECTIVES

29. To develop a web-based automobile knowledge assistant.
30. To support automobile PDF document upload and management.
31. To extract text from digital and scanned documents.
32. To divide documents into meaningful searchable chunks.
33. To generate and store vector embeddings.
34. To implement semantic and keyword-based retrieval.
35. To improve retrieval quality using reranking.
36. To generate grounded answers using Gemini.
37. To provide document and page-level source references.
38. To maintain user conversations and knowledge-base information in a structured database.

7. SCOPE OF THE PROJECT

- Automobile manuals and technical reference documents.
- Maintenance and troubleshooting information.
- Vehicle components and operating information.
- Automotive specifications available in the uploaded knowledge base.
- Document-based question answering.
- Source-aware responses.
- Document upload, update, reprocessing, and deletion.
- Future expansion to multilingual and voice-based assistance.

8. KEY FEATURES

Feature	Description
Document Upload	Upload automobile PDFs through the web interface.
Text Extraction	Extract text and page information from documents.
OCR	Process scanned/image-based PDFs when normal extraction is insufficient.
Chunking	Split extracted text into manageable retrieval units.
Embeddings	Convert chunks into vector representations.
Vector Search	Retrieve semantically relevant document chunks.
Keyword Search	Find exact or textually related information.
Hybrid Retrieval	Combine semantic and keyword results.
Reranking	Rank candidate chunks according to query relevance.
Gemini Generation	Generate natural-language answers using retrieved context.
Citations	Display actual document and page references.
Chat History	Save and reopen previous conversations.
Knowledge Base	Manage indexed documents and processing status.
Authentication	Provide secure user access using Firebase Authentication.
Admin Management	Allow authorized administrators to manage users and documents.

9. SYSTEM ARCHITECTURE

The high-level architecture is:

User → React Frontend → FastAPI Backend → Document/RAG Services → PostgreSQL + pgvector → Gemini → Answer + Sources

Detailed architecture:

- Presentation Layer: React + Vite web interface.
- API Layer: FastAPI REST endpoints.
- Authentication Layer: Firebase Authentication and backend token verification.
- Document Processing Layer: PDF extraction, OCR, cleaning, metadata extraction, and chunking.
- Retrieval Layer: PostgreSQL full-text/keyword search, pgvector semantic search, hybrid scoring, and reranking.
- Generation Layer: Gemini LLM using retrieved context.
- Persistence Layer: PostgreSQL + pgvector as the single persistent database.
- Citation Layer: Maps generated responses to the actual retrieved chunks, documents, and page numbers.

10. TECHNOLOGY STACK

Layer	Technology	Purpose
Frontend	React + Vite	Web application interface
Styling	Tailwind CSS	Responsive UI design

Backend	FastAPI	REST API and application logic
Database	PostgreSQL	Persistent relational data
Vector Extension	pgvector	Embedding storage and similarity search
Authentication	Firebase Authentication	User authentication
LLM	Google Gemini	Grounded answer generation
Embeddings	Hugging Face Sentence Transformers	Vector representation of chunks
Reranking	CrossEncoder	Candidate relevance ranking
PDF Processing	PyMuPDF	PDF text extraction
OCR	Tesseract	Scanned document text recognition
API Communication	Axios / REST	Frontend-backend communication
Deployment	Vercel + Render	Frontend and backend hosting

11. PROJECT MODULES

11.1 Authentication Module

Handles registration, login, logout, Firebase token verification, protected routes, and role-based access.

11.2 Document Management Module

Handles PDF upload, listing, details, deletion, reprocessing, and processing status.

11.3 Document Processing Module

Extracts text, tracks pages, performs OCR fallback, cleans text, and prepares content for chunking.

11.4 Chunking Module

Splits document text into overlapping chunks while preserving document and page metadata.

11.5 Embedding Module

Generates vector embeddings for each chunk using a configurable sentence-transformer model.

11.6 Retrieval Module

Performs semantic vector retrieval and PostgreSQL keyword/full-text search.

11.7 Hybrid Search Module

Combines semantic and keyword retrieval results using configurable weights.

11.8 Reranking Module

Uses a CrossEncoder to improve the order of retrieved candidate chunks.

11.9 LLM Module

Sends the user query and retrieved context to Gemini and generates a grounded answer.

11.10 Citation Module

Attaches actual document, page, chunk, and relevance information to the response.

11.11 Chat Module

Creates conversations, stores messages, retrieves history, and manages multi-turn interaction.

11.12 Knowledge Base Module

Displays indexed documents, chunk counts, processing states, and search functionality.

11.13 Admin Module

Provides authorized administrative operations for documents, users, and system statistics.

12. DATABASE DESIGN

The project uses one PostgreSQL database. The pgvector extension allows the same database to store vector embeddings alongside normal relational records. This simplifies the architecture and makes the system easier to explain and maintain.

Table	Important Fields	Purpose
users	id, firebase_uid, email, name, role, created_at	Stores authenticated users.
documents	id, uploaded_by, filename, title, status, pages, chunks	Stores document metadata and processing state.
document_chunks	id, document_id, chunk_index, page_number, content, embedding	Stores searchable text chunks and vectors.
conversations	id, user_id, title, created_at	Stores chat sessions.
messages	id, conversation_id, role, content, created_at	Stores user and assistant messages.
message_sources	id, message_id, chunk_id, relevance_score, citation_order	Connects answers to actual retrieved sources.

Relationship summary:

- One user can upload many documents.
- One document contains many document chunks.
- One user can create many conversations.
- One conversation contains many messages.
- One assistant message can have many source references.
- Each source reference points to an actual document chunk.

13. RAG WORKFLOW

39. User uploads an automobile document.
40. The backend stores document metadata.
41. The system extracts text page by page.

42. OCR is triggered when normal extraction is insufficient.
43. Text is cleaned and divided into chunks.
44. Embeddings are generated for each chunk.
45. Chunks and embeddings are stored in PostgreSQL + pgvector.
46. The user enters a question in the chat interface.
47. The query is converted into an embedding.
48. Vector search retrieves semantically similar chunks.
49. Keyword search retrieves textually relevant chunks.
50. The results are combined using hybrid retrieval.
51. A CrossEncoder reranks the candidates.
52. The best chunks are selected as context.
53. The context and question are passed to Gemini.
54. Gemini generates a grounded answer.
55. The system attaches actual database citations.
56. The answer, sources, and conversation are stored.
57. The frontend displays the response and sources.

14. DOCUMENT MANAGEMENT WORKFLOW

The uploaded document page shown in the current interface provides the document-management entry point.

58. Open the Documents page.
59. Select the Upload button under 'Upload Automobile PDFs'.
60. Choose an automobile PDF from the local system.
61. Validate the file type and size.
62. Send the file to the backend upload endpoint.
63. Create a document record with PROCESSING status.
64. Extract text and metadata.
65. Perform OCR if required.
66. Create chunks and embeddings.
67. Store chunks and vectors in the database.
68. Update the document status to PROCESSED.
69. Display the indexed document in the 'Indexed Documents' section.

15. SEARCH AND QUESTION ANSWERING WORKFLOW

Example query: 'What are the common causes of engine overheating?'

70. The user submits the question.
71. The backend validates the authenticated request.
72. The query is converted into an embedding.
73. The database performs vector similarity search.
74. PostgreSQL keyword search identifies exact terms and related text.

75. Both result sets are combined.
76. The CrossEncoder reranks candidate chunks.
77. The top relevant chunks are selected.
78. The system constructs a context-aware prompt.
79. Gemini receives only the selected context and user question.
80. The model generates a concise answer.
81. The system attaches real document and page references.
82. The answer and sources are displayed in the chat interface.

16. FRONTEND DESIGN AND CURRENT INTERFACE

The provided screenshot shows the current Document Management interface of the Automobile Knowledge Assistant. The interface uses a dark theme with a left navigation sidebar and a main document-management workspace.

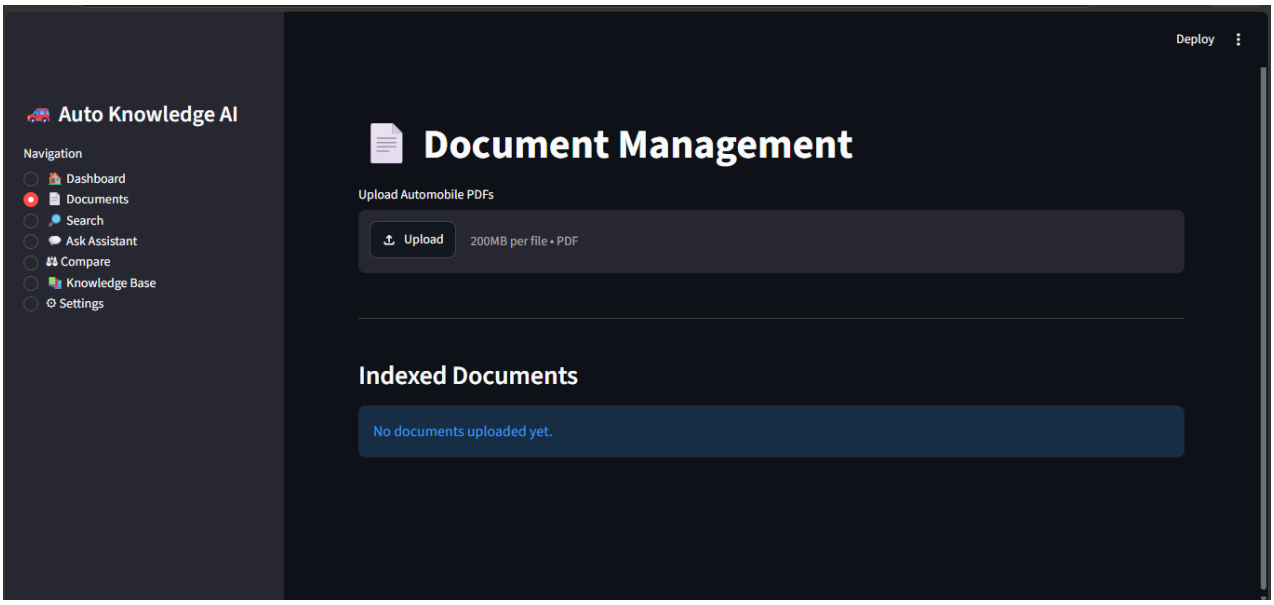


Figure 1: Current Document Management Interface

Visible interface elements:

- Application branding: 'Auto Knowledge AI'.
- Navigation options: Dashboard, Documents, Search, Ask Assistant, Compare, Knowledge Base, and Settings.
- Main heading: Document Management.
- Upload area labeled 'Upload Automobile PDFs'.
- Upload button for selecting automobile PDF files.
- File limit information displayed as 200MB per file and PDF.
- Indexed Documents section.
- Empty-state message: 'No documents uploaded yet.'
- Deploy control is visible in the application environment.

Recommended next implementation step for this page: after a successful upload, replace the empty-state message with real document cards containing filename, status, page count, chunk count, upload date, and actions such as View, Reprocess, and Delete.

17. API DESIGN

Method	Endpoint	Purpose
GET	/api/health	Check backend, database, and AI service health.
GET	/api/auth/me	Return authenticated user information.
POST	/api/documents/upload	Upload and begin processing a document.
GET	/api/documents	List indexed documents.
GET	/api/documents/{id}	View document details.
DELETE	/api/documents/{id}	Delete a document and related chunks.
POST	/api/documents/{id}/reprocess	Reprocess an existing document.
POST	/api/search	Search the automobile knowledge base.
POST	/api/chat	Ask a question and receive a grounded answer.
GET	/api/chat/conversations	List conversations.
GET	/api/chat/conversations/{id}	Get a conversation and messages.
DELETE	/api/chat/conversations/{id}	Delete a conversation.
GET	/api/knowledge/stats	Return knowledge-base statistics.
GET	/api/knowledge/search	Search indexed documents/chunks.

18. SECURITY AND DATA PROTECTION

- Store API keys only in environment variables.
- Never commit .env files containing real credentials.
- Verify Firebase ID tokens in the backend.
- Protect admin endpoints using server-side role checks.
- Validate uploaded file types and sizes.
- Avoid exposing backend stack traces to users.
- Use parameterized/database ORM operations.
- Do not allow the LLM to invent source references.
- Return only authorized users' conversations and documents.

19. ERROR HANDLING

Condition	Expected System Response
Invalid file	Reject upload with a clear validation message.
Unsupported PDF	Show processing failure and log technical details.
Scanned PDF	Attempt OCR before marking processing as failed.

Embedding failure	Keep document status FAILED and show retry option.
Database unavailable	Return service-unavailable response and log error.
Gemini failure/rate limit	Show a temporary AI service error and allow retry.
No relevant results	Tell the user that the knowledge base does not contain enough information.
Unauthorized request	Return HTTP 401/403 and do not expose protected data.

20. TESTING STRATEGY

Testing should cover individual modules as well as the complete end-to-end workflow.

Test Area	Test Case	Expected Result
Authentication	Valid login	User is authenticated and redirected to dashboard.
Upload	Valid PDF	Document is accepted and processing begins.
Validation	Non-PDF/oversized file	Upload is rejected with a useful message.
Extraction	Text PDF	Text and page numbers are extracted.
OCR	Scanned PDF	OCR extracts readable text.
Chunking	Processed document	Chunks are created with metadata.
Embedding	Valid chunk	Vector is generated and stored.
Vector Search	Relevant question	Relevant chunks are returned.
Hybrid Search	Keyword + semantic query	Combined results are returned.
Reranking	Candidate chunks	Relevant results move toward the top.
RAG	Known question	Grounded answer is generated.
Citation	Answer with sources	Real document/page sources are shown.
History	Previous conversation	Conversation can be reopened.
Admin	Unauthorized user	Admin endpoint is blocked.

21. EXPECTED RESULTS

The completed system should allow a user to upload automobile documents and ask natural-language questions about their content. The system should retrieve relevant passages from the uploaded knowledge base, generate a concise answer using Gemini, and display real source references. The Documents page should show processed files instead of the initial empty state, while the Knowledge Base and Chat pages should expose the indexed information and AI responses.

22. ADVANTAGES

- Combines document retrieval and generative AI in one application.
- Uses a single PostgreSQL database for structured data and embeddings.
- Provides semantic as well as keyword search.

- Reranking improves retrieval relevance.
- Source references improve transparency and verification.
- Supports scanned PDFs through OCR.
- Documents can be updated and reprocessed.
- Conversation history improves usability.
- Architecture is modular and suitable for future expansion.
- Can be deployed as a web application.

23. LIMITATIONS

- Answer quality depends on the quality and coverage of uploaded documents.
- OCR accuracy may decrease for low-quality scans.
- Embedding and reranking models require computational resources.
- Gemini API usage may be subject to quota, latency, or cost limits.
- The current knowledge base is restricted to documents uploaded to the application.
- The system should not be treated as a substitute for professional mechanical diagnosis or safety-critical instructions.

24. FUTURE ENHANCEMENTS

- Multilingual automobile question answering.
- Voice input and speech output.
- Image-based automobile component recognition.
- Vehicle-specific knowledge profiles.
- Comparison of specifications across vehicles.
- Automatic document versioning.
- Cloud object storage for large document collections.
- Streaming Gemini responses.
- Advanced evaluation metrics for RAG quality.
- Feedback-based retrieval improvement.
- Mobile application.
- Role-based institutional knowledge management.

25. DEPLOYMENT PLAN

83. Create a PostgreSQL database with pgvector enabled.
84. Configure backend environment variables.
85. Configure Firebase Authentication.
86. Deploy FastAPI backend to Render or another Python-compatible platform.
87. Configure the backend DATABASE_URL and GEMINI_API_KEY.
88. Deploy the React frontend to Vercel.

89. Set the frontend API base URL to the deployed backend.
90. Configure CORS for the frontend domain.
91. Test login, upload, indexing, search, RAG, and citations in the deployed environment.

26. PROJECT STRUCTURE

```
ai-automobile-assistant/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── context/
│   │   ├── hooks/
│   │   ├── App.jsx
│   │   └── main.jsx
│   └── package.json
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── core/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   ├── rag/
│   │   ├── document_processing/
│   │   └── utils/
│   ├── tests/
│   ├── requirements.txt
│   └── .env.example
├── database/
│   ├── schema.sql
│   └── seed.sql
├── docs/
├── sample_documents/
├── docker-compose.yml
├── .gitignore
└── README.md
```

27. CONCLUSION

The AI-Powered Automobile Knowledge Assistant provides an end-to-end approach for building a domain-specific Retrieval-Augmented Generation system. The project combines document processing, OCR, embeddings, vector search, keyword search, reranking, Gemini-based generation, citations, authentication, and conversation management in a single web application. The use of PostgreSQL with pgvector keeps the persistent data architecture simple by storing both conventional application records and vector embeddings in one database.

The current Document Management interface already establishes the foundation for document ingestion. The next major implementation step is to connect the Upload button to the real backend

ingestion pipeline and populate the Indexed Documents section from the database. Once that is integrated with retrieval and Gemini, the application becomes a complete automobile-focused RAG assistant suitable for a mini-project demonstration and future portfolio development.

28. VIVA QUESTIONS AND ANSWERS

Q: What is RAG?

A: Retrieval-Augmented Generation combines information retrieval with an LLM so the model can generate answers using retrieved external context.

Q: Why is RAG used?

A: It helps the system answer using the project's specific documents and reduces dependence on the model's general knowledge.

Q: Why use embeddings?

A: Embeddings represent text as vectors so semantically similar questions and document chunks can be matched.

Q: Why use pgvector?

A: It allows vector embeddings and similarity search to be handled inside PostgreSQL, satisfying the single-database design.

Q: What is hybrid search?

A: It combines semantic/vector search with keyword or full-text search.

Q: Why use reranking?

A: Reranking improves the order of retrieved candidates so the most relevant chunks are passed to the LLM.

Q: Why is page metadata important?

A: It allows the system to provide verifiable page-level citations.

Q: What happens if no relevant information is found?

A: The assistant should clearly state that the current knowledge base does not contain enough information rather than inventing an answer.

Q: Why use OCR?

A: OCR extracts text from scanned or image-based PDF pages that do not contain machine-readable text.

Q: What is the role of Gemini?

A: Gemini generates the final natural-language answer using the retrieved context.

Q: Why use Firebase?

A: Firebase Authentication provides a convenient way to authenticate users and issue identity tokens.

Q: What is the single-database design?

A: PostgreSQL stores users, documents, chunks, embeddings, conversations, messages, and source mappings in one database.

29. IMPLEMENTATION CHECKLIST

- Connect the current Documents → Upload button to POST /api/documents/upload.
- Create the PostgreSQL + pgvector schema.
- Implement PDF extraction and OCR fallback.
- Implement chunking with page metadata.
- Generate and store embeddings.
- Implement vector and keyword retrieval.
- Implement hybrid search and CrossEncoder reranking.
- Connect Gemini to the RAG pipeline.
- Return real citations from database chunks.
- Populate the Indexed Documents section from the backend.
- Implement Search and Ask Assistant pages.
- Implement conversation history.
- Connect Firebase authentication.
- Add Admin controls.
- Add automated tests.
- Complete README and deployment configuration.